

🔄 Workflow Sequence

Setup → Brain (100%) → Audit → Triage → Fix → Test → Learn → Report

PHASE	COMMAND	DESCRIPTION
Setup	<code>/setup-new-client</code>	New project — creates dirs, config, GitHub labels
	<code>/setup-existing-client</code>	Existing client — copies brain, diffs for divergence
Build	<code>/build-module-brain</code>	Run until COVERAGE_TRACKER = 100%. Reads all code → 8–9 brain docs
	<code>/manage-bug-patterns</code>	Init / scan / add pattern entries
Find	<code>/module-bug-audit</code>	6 rounds: Controller → Schema → Model → JS → Calcs → AJAX
Classify	<code>/bug-intake-triage</code>	Severity + Track + Category + Sprint + GitHub Issue created
Fix	<code>/fix-single-bug</code>	Entry point for ALL fixes. Routes based on track/complexity
	<code>/fix-architecture-bug</code>	Track A (system) — Antigravity leads, human approves diff
	<code>/fix-business-bug</code>	Track B (business) — human validates rule before code changes
Verify	<code>/test-and-verify</code>	Syntax → category tests → smoke test (auto-called by fix workflows)
Close	<code>/learn-and-improve</code>	Updates brain + patterns + GitHub + ACTIVE_BUGS + velocity
Report	<code>/sprint-status</code>	Sprint progress from GitHub + ACTIVE_BUGS
	<code>/system-health</code>	Cross-module risk heatmap 🟡/🟠/🟢
	<code>/overall-bug-dashboard</code>	Full report + developer attribution + velocity trends
Maintain	<code>/validate-workflows</code>	Checks files exist, cross-refs valid, no hardcoded paths

✅ DOs

1. **Brain before audit** — no brain = missed bugs
2. **Triage before fix** — every bug needs severity, track, sprint, GitHub issue
3. **Always enter via /fix-single-bug** — it has guards, routing, and tracking
4. **Trace full data flow** for money bugs: Form → JS → AJAX → PHP → DB
5. **Check Module Brain first** before reading source files
6. **Run /learn-and-improve after every fix** — not optional
7. **Document rollback BEFORE applying fix** → ROLLBACK_REGISTRY.md
8. **Check for duplicates** before creating bug entries
9. **Syntax check after every PHP change**
10. **Keep ACTIVE_BUGS.md ↔ GitHub in sync**
11. **Run brain building until COVERAGE_TRACKER = 100%**

❌ DON'Ts

1. **Don't audit without brain** — you'll miss context-dependent bugs
2. **Don't skip triage** — no tracking = chaos
3. **Don't fix without root cause** — symptom patches create new bugs
4. **Don't call fix sub-workflows directly** — go through /fix-single-bug
5. **Don't apply fixes without human approval** on Track A/B
6. **Don't skip /learn-and-improve** — it's how the system learns
7. **Don't duplicate functions** — extend existing ones
8. **Don't run destructive SQL** without approval
9. **Don't hardcode paths** — use config variables
10. **Don't fix one layer only** — if JS is wrong, check PHP too



Templates

Located in knowledge_brain/_TEMPLATE/

COVERAGE_TRACKER.md

Brain completeness per metric — **must reach 100%** before audit

Used by: /build-module-brain

FORENSIC_TEMPLATE.md

6-layer investigation: Symptom → Reproduce → JS → PHP → DB → Root Cause

Used by: /fix-architecture-bug, /fix-business-bug

INVARIANT_MATRIX.md

Variant × behavior grids (e.g., Scheme Type × GST Type)

Used by: /build-module-brain, /fix-business-bug



Bug Classification

SEVERITY	CRITERIA	SPRINT	TRACK	MEANING	WHO LEADS
P0	Money wrong, data lost, security hole	Sprint 1	Track A	System — code defect, runs wrong	Antigravity leads
P1	Shows wrong info, blocks workflow	Sprint 2	Track B	Business — wrong business result	Human validates rule
P2	Works but wrong behavior	Sprint 3	LogicData ValidationDatabaseException HandlingSecurityPerformanceIntegrationConcurrency		
P3	Cosmetic, edge case	Sprint 4			



Human Checkpoints

WORKFLOW	STEP	WHAT YOU DO
/fix-architecture-bug	Step 5	Approve proposed fix approach
/fix-architecture-bug	Step 10	Review final diff
/fix-business-bug	Step 5	Confirm the correct business rule
/fix-business-bug	Step 10	Smoke test with real data
/fix-single-bug	Step 7	Approve diff + test results



Key Files

.agent/config.md — Central config (all workflows)	.agent/GEMINI.md — Project rules
ACTIVE_BUGS.md — Live bug tracker	ROLLBACK_REGISTRY.md — Emergency rollback index
COMMON_BUG_PATTERNS.md — Pattern library + detection	MODULE_BRAIN.md — Module architecture + risks